

Programming Logic in Python

From Symbolic Reasoning to Rule-Based Systems

A comprehensive guide for teaching computational logic, formal reasoning, and AI-driven inference using Python

Why Study Logic in Python? 🎯

Foundation of AI 🤖

Logic forms the bedrock of reasoning and inference in artificial intelligence systems, enabling machines to make decisions based on formal rules.

Decision Automation ⚙️

Automate complex decision-making processes in **expert systems**, **theorem provers**, and **intelligent agents**.

Practical Applications 💼

Used across domains: search algorithms, knowledge representation, constraint solving, and business rule engines.



Learning Outcomes 📚

By the end of this session, you will master the following competencies:

01

Understand Logical Reasoning in Python

Grasp how formal logic translates into executable code and computational models.

02

Master Logic Libraries

Use **SymPy**, **Kanren**, **PyDatalog**, **Pytholog**, and **jsonLogic** for different reasoning paradigms.

03

Build Reasoning Systems

Implement inference engines, knowledge bases, and rule-based decision systems from scratch.

04

Compare Paradigms

Evaluate symbolic vs. declarative vs. data-driven approaches to logic programming.

What is Logic Programming? 🔍

Logic programming is a declarative paradigm where you specify *what* is true rather than *how* to compute it. Programs consist of:

- **Facts:** Basic truths about the world
- **Rules:** Logical implications and relationships
- **Queries:** Questions about what can be inferred

The system automatically performs logical inference to derive conclusions from facts and rules.

Prolog-Style Example 📝

```
parent(adam, bob).
parent(bob, charlie).

grandparent(X, Z) :-
 parent(X, Y),
 parent(Y, Z).
```

Types of Logic in Python 🐍

ƒ(×)

Symbolic Logic

Description: Boolean formulas, propositional calculus, and automated theorem proving

Libraries: SymPy

Use Case: SAT solving, proof verification, tautology detection

Logic Programming

Description: Relational reasoning with facts and queries

Libraries: Kanren, PyDatalog, Pytholog

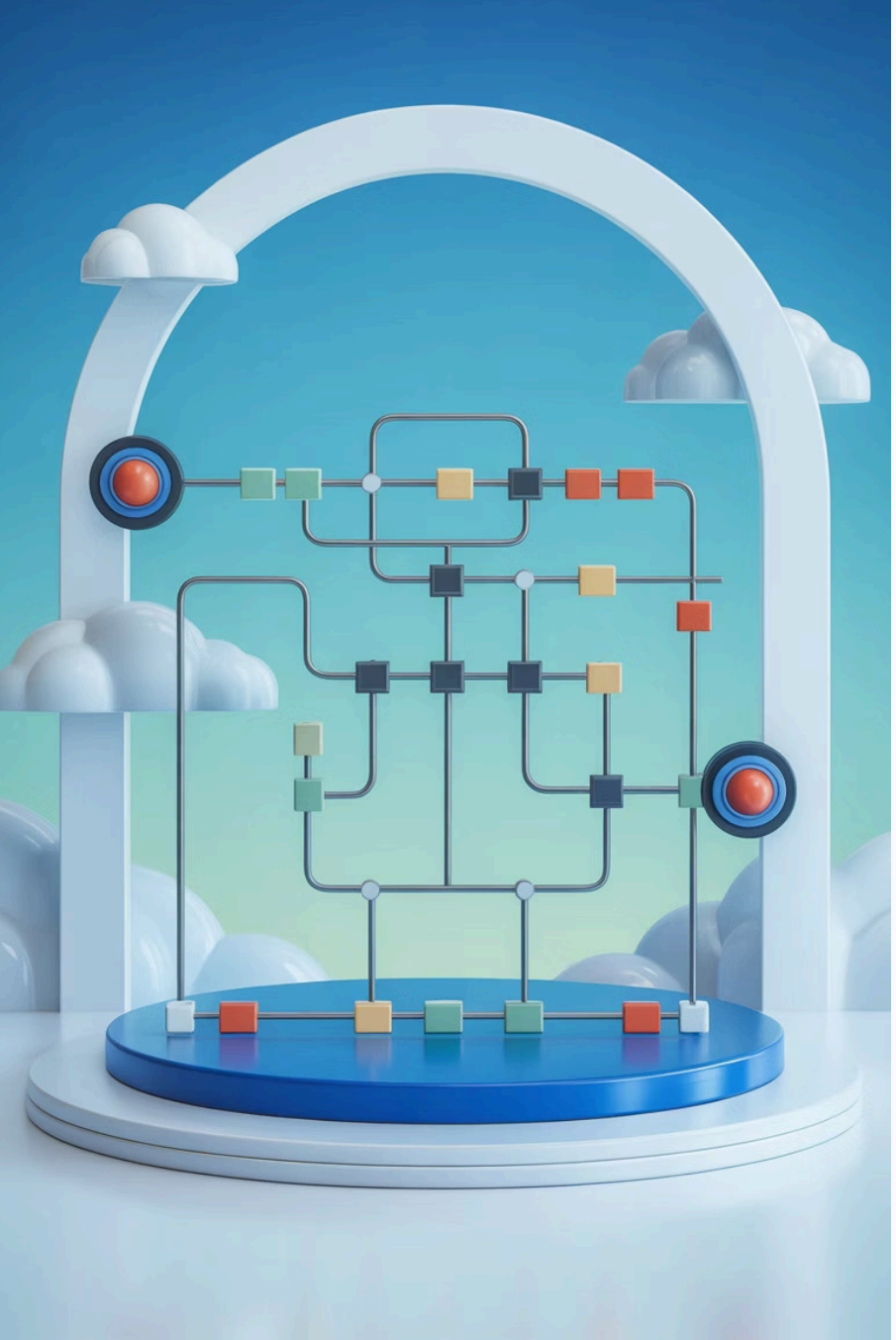
Use Case: Knowledge bases, expert systems, AI agents

Rule-Based Logic

Description: JSON-encoded conditional rules

Libraries: jsonLogic

Use Case: Dynamic business logic, API-driven decisions



Propositional Logic Foundations ⚡

Formal Definition

Propositional logic uses Boolean variables and logical connectives:

- P, Q, R – atomic propositions
- $\neg P$ – negation (NOT)
- $P \wedge Q$ – conjunction (AND)
- $P \vee Q$ – disjunction (OR)
- $P \rightarrow Q$ – implication (IF-THEN)
- $P \leftrightarrow Q$ – biconditional (IFF)

Truth Values

Each proposition evaluates to **True** or **False**. Complex formulas combine propositions using connectives.

$$(P \wedge Q) \rightarrow R$$

This formula reads: "If both P and Q are true, then R must be true."

SymPy for Symbolic Logic 🔢

SymPy enables symbolic reasoning with propositional formulas. Let's explore key operations:

```
from sympy import symbols, And, Or, Not, Implies, Equiv
from sympy.logic.boolalg import simplify_logic
from sympy.logic.inference import satisfiable

# Define propositions
P, Q, R = symbols('P Q R')

# Build a formula: (P ∧ Q) → R
formula = Implies(And(P, Q), R)

# Simplify the expression
simplified = simplify_logic(formula)
print(f"Simplified: {simplified}")  # ¬P ∨ ¬Q ∨ R

# Check satisfiability (find truth assignment)
model = satisfiable(formula)
print(f"Satisfiable: {model}")  # {P: False} or similar
```

Key Insight: The implication $P \wedge Q \to R$ is equivalent to $\neg P \vee \neg Q \vee R$ by logical equivalence laws. ✨

Truth Tables with SymPy 📊

Generating Truth Tables

```
from sympy.logic.boolalg import truth_table

# Formula: P → Q
formula = Implies(P, Q)

# Generate truth table
for row in truth_table(formula, [P, Q]):
    print(row)
```

Output displays all combinations of truth values and the formula's result.

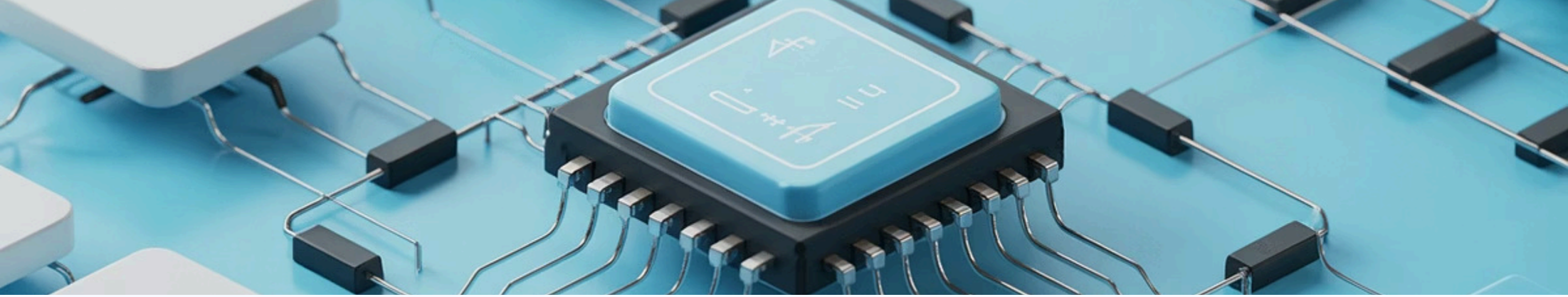
Logical Equivalence

Two formulas ϕ and ψ are equivalent if:

$$\phi \leftrightarrow \psi \text{ is a tautology}$$

```
expr1 = Implies(P, Q)
expr2 = Or(Not(P), Q)

# Check equivalence
equiv = Equiv(expr1, expr2)
print(simplify_logic(equiv)) # True
```



SAT Solving and Model Checking 🎲

A formula is satisfiable if there exists at least one truth assignment that makes it true.

$$\text{SAT}(\phi) = \exists v_1, v_2, \ldots, v_n : \phi(v_1, v_2, \ldots, v_n) = \text{True}$$

```
from sympy.logic.inference import satisfiable

# Complex formula
formula = And(Or(P, Q), Or(Not(P), R), Or(Not(Q), Not(R)))

# Find satisfying assignment
result = satisfiable(formula)
if result:
    print(f"SAT: {result}")  # e.g., {P: True, Q: False, R: True}
else:
    print("UNSAT: No solution exists")
```

Applications: Circuit verification, constraint solving, automated planning 🚀

SymPy Use Cases 

Theorem Proving

Verify mathematical proofs and detect tautologies – formulas that are always true.

Contradiction Detection

Identify inconsistent knowledge bases where formulas cannot be simultaneously true.

Circuit Verification

Model digital circuits as Boolean formulas and verify correctness using SAT solvers.

Teaching Tool

Interactive demonstrations of propositional logic, truth tables, and logical equivalences for students.

First-Order Logic (FOL) 🌐

First-order logic extends propositional logic with **quantifiers** and **predicates** over objects:

Syntax Elements

- **Variables:** x, y, z
- **Constants:** a, b, c
- **Predicates:** $P(x)$, $Parent(x, y)$
- **Functions:** $f(x)$, $mother(x)$
- **Quantifiers:**

$$\forall x\, P(x) \quad \text{(universal)}$$

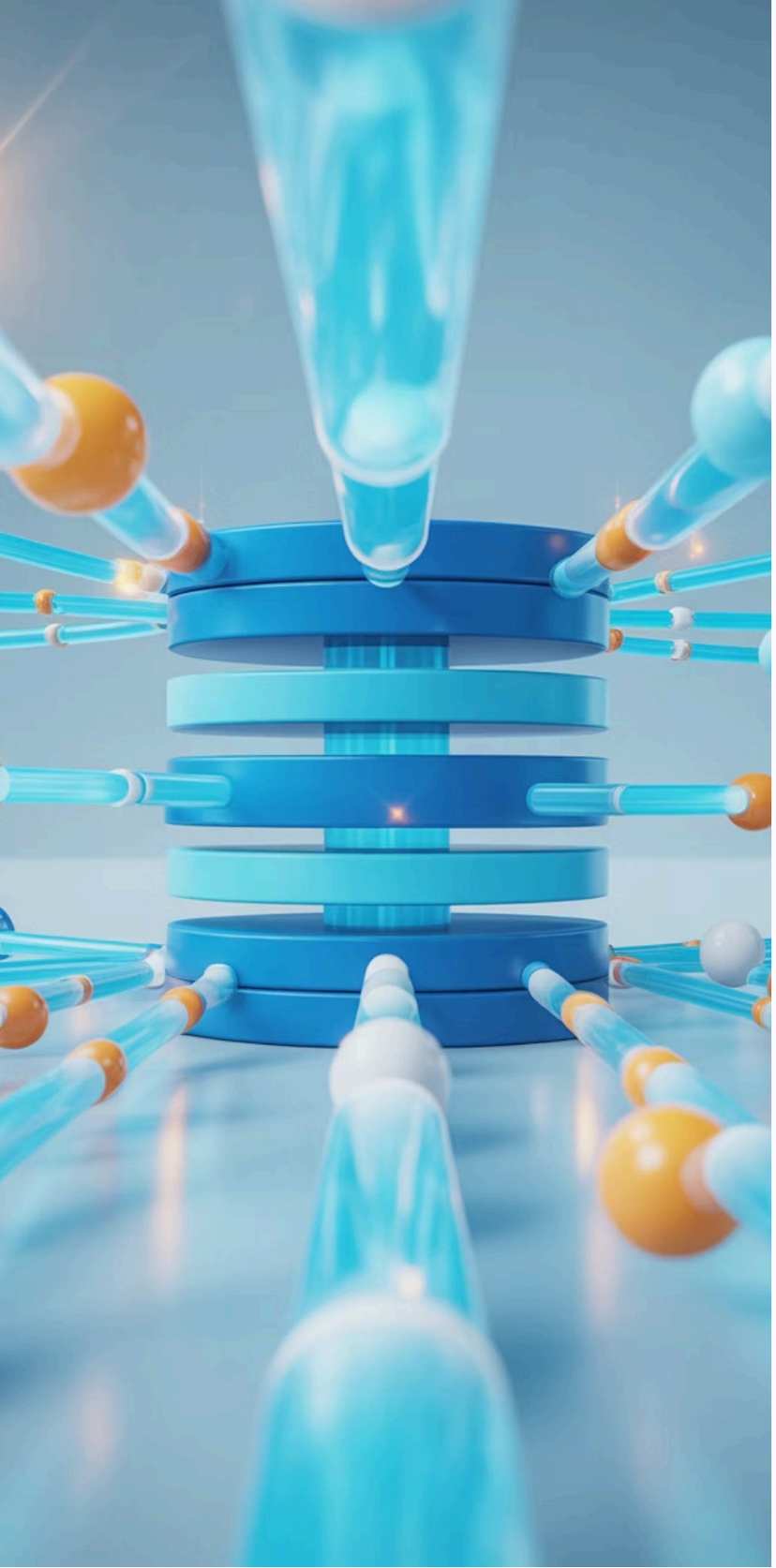
$$\exists x\, P(x) \quad \text{(existential)}$$

Example: Family Relations

$$\forall x, y\, (\text{Parent}(x, y) \rightarrow \text{Ancestor}(x, y))$$

$$\forall x, y, z\, (\text{Parent}(x, y) \land \text{Parent}(y, z) \rightarrow \text{Grandparent}(x, z))$$

These formulas define recursive relationships in a family tree. 👨‍👩‍👧‍👦



Logic Programming with Kanren 🔗

Kanren implements relational programming – similar to Prolog but in Python.

```
from kanren import run, var, Relation, facts, eq

# Define a relation for parent-child
parent = Relation()
facts(parent, ("Adam", "Bob"),
              ("Bob", "Charlie"),
              ("Adam", "Diana"))

# Query: Who are Adam's children?
x = var()
result = run(0, x, parent("Adam", x))
print(result)  # ('Bob', 'Diana')

# Query: Who is Charlie's parent?
y = var()
result = run(0, y, parent(y, "Charlie"))
print(result)  # ('Bob',)
```

Formal Semantics: A query $\exists x \, \text{parent}(\text{Adam}, x)$ finds all x satisfying the relation. 🎯

Recursive Rules in Kanren 🔄

Define transitive relations like ancestor using recursion:

```
from kanren import conde, lall

# Base case: parent is ancestor
def ancestor(x, y):
    return conde([parent(x, y)],
                 [lall([parent(x, z), ancestor(z, y)])])

# Query: Who are Adam's descendants?
z = var()
result = run(0, z, ancestor("Adam", z))
print(result)  # ('Bob', 'Charlie', 'Diana')
```

This encodes the logical rule:

$$\text{Ancestor}(x, y) \leftarrow \text{Parent}(x, y)$$

$$\text{Ancestor}(x, y) \leftarrow \exists z \, (\text{Parent}(x, z) \land \text{Ancestor}(z, y))$$

The system performs **backward chaining** to find all solutions! ✨

PyDatalog: SQL Meets Logic 🗄️

PyDatalog combines Datalog (a subset of Prolog) with Python for querying relational data.

```
from pyDatalog import pyDatalog

pyDatalog.create_terms('X, Y, Z, parent, ancestor')

# Assert facts
+parent('Adam', 'Bob')
+parent('Bob', 'Charlie')
+parent('Diana', 'Eve')

# Define recursive rule
ancestor(X, Y) <= parent(X, Y)
ancestor(X, Y) <= (parent(X, Z) & ancestor(Z, Y))

# Query all ancestors of Charlie
print(ancestor('Adam', Y))  # Bob, Charlie
```

Datalog Syntax

Rules use **Horn clauses**:

$$H \leftarrow B_1 \land B_2 \land \ldots \land B_n$$

Where H is the head and B_i are body literals.

Key Feature: Automatic recursion handling and efficient query optimization. 🚀

PyDatalog for Graph Queries 🕸️

Model graph structures and perform **reachability queries**:

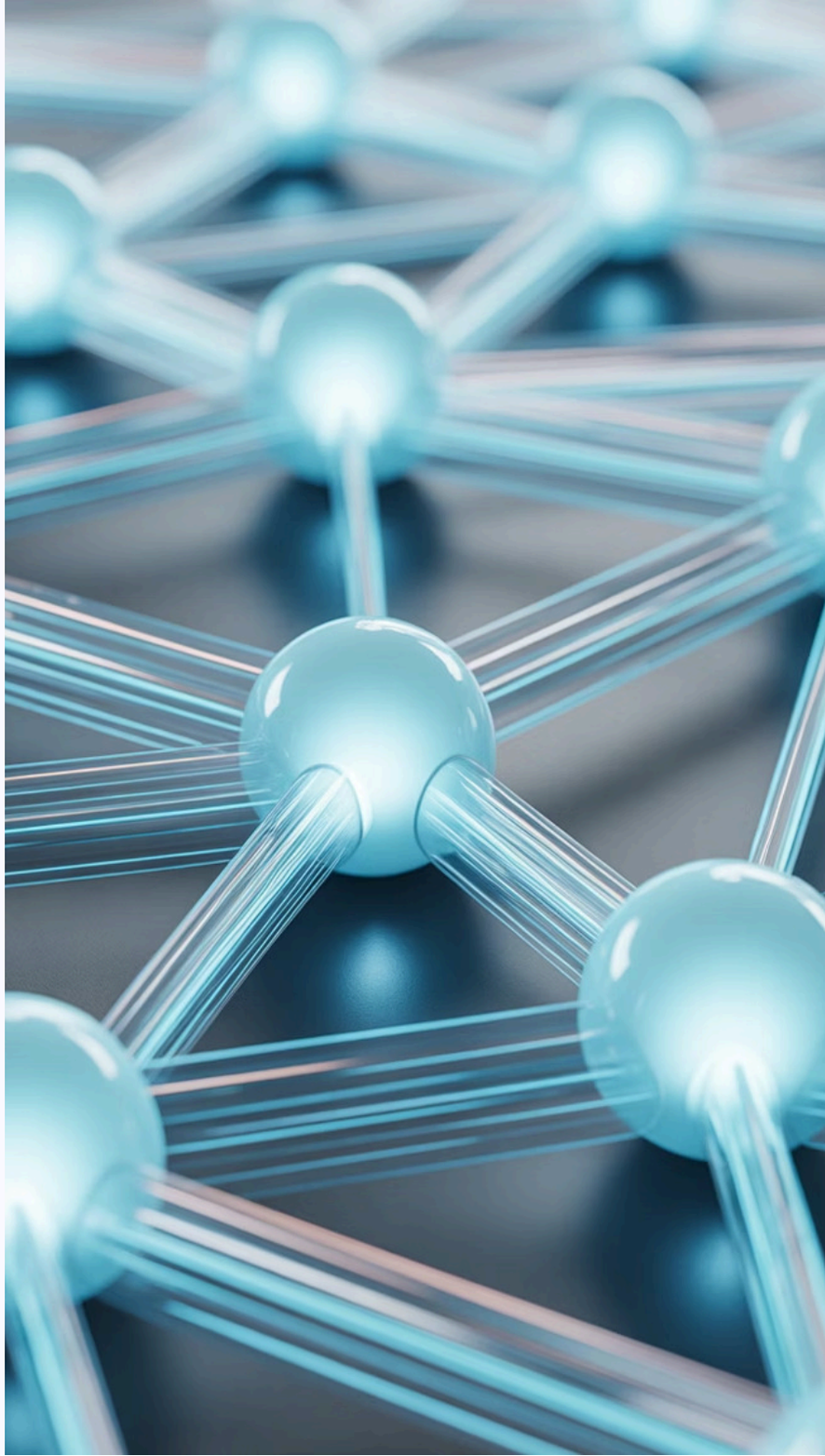
```
pyDatalog.create_terms('edge, reachable, Node, N1, N2, N3')

# Define graph edges
+edge('A', 'B')
+edge('B', 'C')
+edge('C', 'D')
+edge('A', 'E')

# Define reachability
reachable(N1, N2) <= edge(N1, N2)
reachable(N1, N2) <= (edge(N1, N3) & reachable(N3, N2))

# Query: Is D reachable from A?
print(reachable('A', 'D'))  # Yes!
```

This implements transitive closure over graph edges – fundamental for path finding! 🛤️



Pytholog: Prolog in Python 🐍

Pytholog provides a lightweight Prolog-like interface with intuitive syntax:

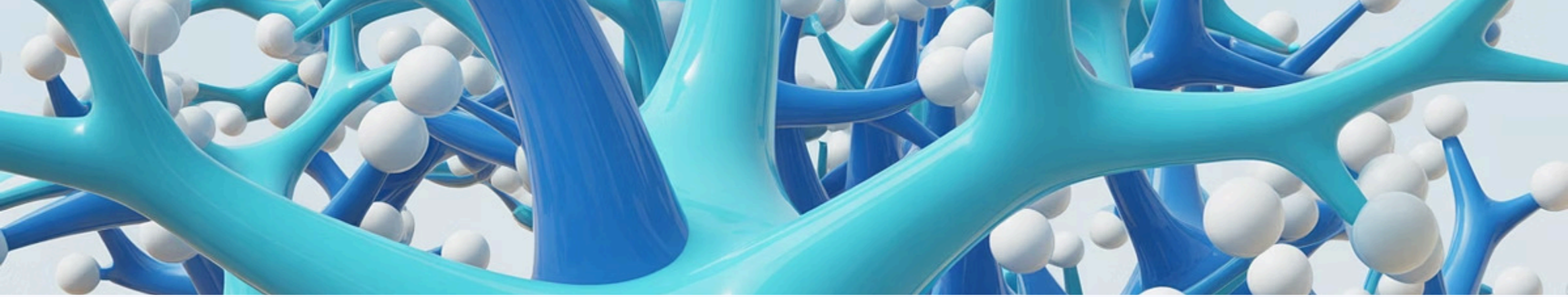
```
import pytholog as pl

# Create knowledge base
kb = pl.KnowledgeBase("family")
kb([
    "parent(adam, bob)",
    "parent(bob, charlie)",
    "parent(diana, eve)",
    "male(adam)", "male(bob)", "male(charlie)",
    "female(diana)", "female(eve)",

    # Rules
    "father(X, Y) :- parent(X, Y), male(X)",
    "mother(X, Y) :- parent(X, Y), female(X)",
    "grandparent(X, Z) :- parent(X, Y), parent(Y, Z)"
])

# Query the knowledge base
print(kb.query(pl.Expr("father(adam, Who)")))  # bob
print(kb.query(pl.Expr("grandparent(adam, charlie)")))  # True
```

Use Case: Expert systems, diagnostic reasoning, family tree queries 👨‍👩‍👧‍👦



Pytholog: Complex Queries 🔎

Multi-Variable Queries

```
# Find all grandfather relationships
kb([
    "grandfather(X, Z) :- father(X, Y), parent(Y, Z)"
])

result = kb.query(pl.Expr("grandfather(X, Z)"))
for binding in result:
    print(f"{binding['X']} is grandfather of {binding['Z']}")
```

Negation as Failure

```
# Define unmarried
kb([
    "married(adam, diana)",
    "single(X) :- male(X), not married(X, _)"
])

print(kb.query(pl.Expr("single(bob)")))  # True
```

Uses closed-world assumption: what cannot be proven is false.

Rule-Based Systems with jsonLogic ⚙️

jsonLogic represents conditional logic as JSON data – perfect for APIs and dynamic rules:

Basic Example

```
from json_logic import jsonLogic

# Rule: age > 18 AND score < 100
rule = {
   "and": [
      {">": [{"var": "age"}, 18]},
      {"<": [{"var": "score"}, 100]}
   ]
}

data = {"age": 25, "score": 85}
result = jsonLogic(rule, data)
print(result)  # True
```

Formal Semantics

The rule encodes:

$$(\text{age} > 18) \land (\text{score} < 100)$$

Evaluated against variable bindings from the data object.

Benefit: Non-programmers can edit rules as JSON! 📝



Complex jsonLogic Rules 🎯

Build sophisticated decision trees with nested conditions:

```
rule = {
    "if": [
        {">": [{"var": "temperature"}, 30]}, "hot",
        {">": [{"var": "temperature"}, 20]}, "warm",
        {">": [{"var": "temperature"}, 10]}, "cool",
        "cold"
    ]
}

print(jsonLogic(rule, {"temperature": 25}))  # "warm"
```

This implements a multi-way conditional:

$$f(t) = \begin{cases} \text{hot} & \text{if } t > 30 \\ \text{warm} & \text{if } 20 < t \leq 30 \\ \text{cool} & \text{if } 10 < t \leq 20 \\ \text{cold} & \text{otherwise} \end{cases}$$

jsonLogic in Production 🏭

47%

Reduced Code Changes

Business rules stored as data eliminate deployment cycles for logic updates

3.2x

Faster Iteration

Non-technical stakeholders can modify rules without developer involvement

89%

Cross-Platform

Same JSON rules work across Python, JavaScript, Ruby, and other languages

Real-World Applications:

- 🏥 **Healthcare:** Patient eligibility and risk scoring algorithms
- ⚖️ **Legal AI:** Case filtering and document classification rules
- 💰 **Finance:** Dynamic pricing, discount logic, fraud detection
- 🤖 **AI Systems:** Hybrid reasoning combining neural networks with symbolic rules

Comparing Logic Paradigms 📊

Library	Type	Syntax	Strength	Best Use Case
SymPy	Symbolic	Python expressions	Mathematical proofs	SAT solving, tautology detection
Kanren	Relational	Declarative queries	Relation reasoning	Constraint solving, planning
PyDatalog	Datalog	SQL-like rules	Recursive queries	Knowledge graphs, databases
Pytholog	Prolog-style	Horn clauses	AI reasoning	Expert systems, diagnostics
jsonLogic	Rule engine	JSON objects	Dynamic evaluation	Business rules, API logic

Integrating Multiple Paradigms 🔄

Combine different logic systems for hybrid reasoning:

Symbolic Verification

Use **SymPy** to verify that rules are consistent and don't contain contradictions

Knowledge Storage

Store facts and relations in **PyDatalog** for efficient querying and inference

Runtime Decisions

Apply **jsonLogic** rules at runtime for flexible, data-driven decision making

```
# Example: Verify rules with SymPy, execute with jsonLogic
from sympy import symbols, And, Implies, satisfiable
from json_logic import jsonLogic

# Step 1: Verify rule consistency
P, Q = symbols('P Q')
rule_formula = Implies(P, Q)
assert satisfiable(rule_formula), "Rule is consistent ✓"

# Step 2: Convert to jsonLogic for execution
rule_json = {"if": [{"var": "P"}, {"var": "Q"}, True]}
result = jsonLogic(rule_json, {"P": True, "Q": True})
print(f"Execution result: {result}")
```

Neuro-Symbolic AI 🧠⚡

The future of AI combines neural learning with symbolic reasoning:

Neural Networks

- ✅ Learn from data
- ✅ Handle uncertainty
- ✅ Generalize to new cases
- ❌ Black-box decisions
- ❌ Lack interpretability

Symbolic Logic

- ✅ Explicit reasoning
- ✅ Interpretable rules
- ✅ Guaranteed correctness
- ❌ Brittle to noise
- ❌ Manual rule creation

Hybrid Approach: Neural networks learn patterns, then logic systems ensure decisions satisfy formal constraints. 🎯



Constraint Programming 🧩

Constraint solving extends logic programming with numerical and combinatorial constraints:

```
from ortools.sat.python import cp_model

# Create model
model = cp_model.CpModel()

# Variables: assign teachers to courses
teacher_a = model.NewBoolVar('teacher_a_course1')
teacher_b = model.NewBoolVar('teacher_b_course1')

# Constraint: exactly one teacher per course
model.Add(teacher_a + teacher_b == 1)

# Solve
solver = cp_model.CpSolver()
status = solver.Solve(model)

if status == cp_model.OPTIMAL:
    print(f"Teacher A: {solver.Value(teacher_a)}")
    print(f"Teacher B: {solver.Value(teacher_b)}")
```

Applications: Scheduling, resource allocation, planning problems 📅

Knowledge Graphs and RDF

Semantic Web Technologies

Represent knowledge using **RDF triples**:

$$(\text{subject}, \text{predicate}, \text{object})$$

```
from rdflib import Graph, URIRef, Literal

g = Graph()
adam = URIRef("http://example.org/adam")
bob = URIRef("http://example.org/bob")
parent_of = URIRef("http://example.org/parentOf")

g.add((adam, parent_of, bob))
```

SPARQL Queries

```
query = """
SELECT ?child WHERE {
   ?parent parentOf ?child .
   ?parent name "Adam" .
}
"""
results = g.query(query)
```

SPARQL is a query language for RDF – like SQL for knowledge graphs.

Probabilistic Logic 🎲

Combine logic with probability theory for uncertain reasoning:

```
from problog.program import PrologString
from problog import get_evaluatable

# Probabilistic facts
program = PrologString("""
0.7::rainy.
0.9::umbrella :- rainy.
0.2::umbrella :- \+rainy.

query(umbrella).
""")

# Evaluate probability
result = get_evaluatable().create_from(program).evaluate()
print(f"P(umbrella) = {result[1]}")  # 0.69
```

This encodes:

$$P(\text{umbrella}) = P(\text{rainy}) \cdot P(\text{umbrella}|\text{rainy}) + P(\neg\text{rainy}) \cdot P(\text{umbrella}|\neg\text{rainy})$$

Use Cases: Medical diagnosis, risk assessment, uncertain inference 🏥

Hands-On Exercise 💻

Build a Weather Reasoning System

Task: Implement the following inference rules using two approaches:

Facts

- If it's cloudy, it might rain
- If it rains, the ground is wet
- If the ground is wet, bring an umbrella

Approach 1: SymPy

Model as propositional logic with implications and verify consistency

Approach 2: jsonLogic

Create a rule-based decision system that takes sensor data as input

Challenge Extension 🚀

Add probabilistic reasoning: "Cloudy → 70% chance of rain"



Key Takeaways 🎓

1 Logic Enables Structured AI

Formal reasoning provides guarantees and interpretability that pure machine learning cannot offer.

2 Multiple Paradigms Available

Choose symbolic (SymPy), declarative (Kanren, PyDatalog), or rule-based (jsonLogic) based on your needs.

3 Hybrid Systems Are Powerful

Combine paradigms: logic for verification, neural nets for learning, rules for execution. 🔄

4 Real-World Impact

Logic programming powers expert systems, constraint solvers, knowledge graphs, and neuro-symbolic AI. 💡



Additional Resources 📚

Official Documentation

- **SymPy Logic Module:** docs.sympy.org/latest/modules/logic.html
- **Kanren (logpy):** github.com/logpy/logpy
- **PyDatalog:** pypi.org/project/pyDatalog
- **Pytholog:** github.com/mnoorfawi/pytholog
- **jsonLogic:** jsonlogic.com

Further Learning

- 📖 "Artificial Intelligence: A Modern Approach" (Russell & Norvig)
- 📖 "Prolog Programming for Artificial Intelligence" (Bratko)
- 📹 Stanford CS157: Logic & Automated Reasoning
- 📹 MIT 6.825: Techniques in Artificial Intelligence
- 💻 Kaggle: Logic Programming tutorials

Thank You! 🎉

Questions & Discussion

Let's explore how logic programming can transform your AI projects and research!

📧 Contact

Reach out for collaboration opportunities or questions about course materials

🔬 Lab Projects

Join ongoing research in neuro-symbolic AI and automated reasoning systems

💡 Office Hours

Drop by to discuss logic programming applications in your domain

